ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/376437928

Performance comparison of genetic algorithms with traditional search techniques on the N-Queen Problem

Conference Paper · October 2023

DOI: 10.1109/ICIT59216.2023.10335855

CITATIONS
7

READS
349

9 authors, including:



Omar Kashif Majeed
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
2 PUBLICATIONS **21** CITATIONS
SEE PROFILE



Raja Hashim Ali
KTH Royal Institute of Technology
63 PUBLICATIONS **776** CITATIONS
SEE PROFILE



Nisar Ali
University of Regina
19 PUBLICATIONS **234** CITATIONS
SEE PROFILE



Usama Arshad
National University of Computer and Emerging Sciences
46 PUBLICATIONS **363** CITATIONS
SEE PROFILE

All content following this page was uploaded by Omar Kashif Majeed on 24 May 2024.

The user has requested enhancement of the downloaded file.

2023 International Conference on IT and Industrial Technologies (ICIT)

Performance comparison of genetic algorithms with traditional search techniques on the N-Queen Problem

Omar Kashif Majeed*, Raja Hashim Ali*,†, Ali Zeeshan Ijaz*, Nisar Ali‡, Usama Arshad*, Muhammad Imad*, Said Nabi*, Javaria Tahir§ and Memoona Saleem*

* Faculty of Computer Science & Engineering, GIK Institute of Engg. Sciences & Technology, Topi, Pakistan.
† Dept. of Technology & Software Engineering, University of Europe for Applied Sciences, Berlin, Germany.
‡ Faculty of Electronic Systems Engineering, University of Regina, Regina, Canada.
§ Department of Computer Science, COMSATS University Islamabad campus, Islamabad, Pakistan.
* Corresponding Author: hashim.ali@giki.edu.pk

***Abstract*—This paper presents an analysis of solving the N-Queen problem using a genetic algorithm and compares its performance with traditional search algorithms like breadth-first search (BFS) and depth-first search (DFS). The N-Queen problem is a famous problem in the field of artificial intelligence that has been studied in depth. The genetic algorithm is an optimization algorithm inspired by the process of natural selection and evolution in living organisms. BFS and DFS are classical search algorithms that explore the search space to find a solution. This paper discusses all the main components of the genetic algorithm, such as population generation, fitness function, selection, crossover, and mutation, and also explains how it is used to solve the N-Queen problem. Traditional search methods, i.e., BFS and DFS, are also briefly discussed. The experimental results show that classical searching approaches are better for small-sized problems, but the genetic algorithm outperforms BFS and DFS in computation time for medium and large-sized problems. This paper concludes by analyzing the effect of parameter tuning on the genetic algorithm and suggests possible future work in the field.**

***Index Terms*—N-queen problem, genetic algorithm, Breadth First Search (BFS), Depth First Search (DFS).**

I. Introduction

The N-Queen problem is a classical problem in the field of artificial intelligence [1]. In the game of chess, a queen can move in its row, column, or diagonal from its existing cell both in a forward and in backward direction. The solution to the problem is to place N queens on an N x N chessboard such that no queen is being attacked by another queen. Hence, to determine a valid solution for the N-Queen problem, there should not be two queens in the same row, column, or diagonal. This also means that there should be only one queen in each row, column, and its diagonal in the valid solution.

As a sample N-Queens problem, an 8 x 8 chessboard with a possible placement of eight queens is shown in Figure 1. In the 8 x 8 board, eight queens should be placed on this board such that there are no attacking pairs of queens. This scenario is called the 8-Queen problem, and it is the most commonly demonstrated specific example of the N-Queen problem. From Figure 1, it can clearly be seen that at least one queen on the board is being attacked by at least one other queen, and therefore, this board configuration is not a valid solution for the N-Queen problem.

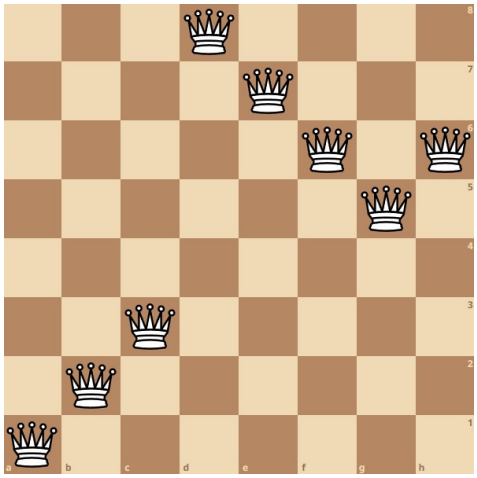


Fig. 1. Example of an invalid board setting for 8-queen problem.

Another 8 x 8 chessboard can be seen in Figure 2. If the board and the position of the queens are carefully analyzed, it can be noticed that none of the eight queens is being attacked by any other queen. Therefore, this board configuration is a valid solution for the 8-Queen problem. It is important to note that this is not the only solution for the 8-Queen problem; In fact, there are 92 solutions for this problem, which is why the 8-Queen problem is considered a medium-sized problem instance.

The N-Queen problem is a classical constraint satisfaction problem that exhibits backtracking behavior and can be solved through backtracking algorithms in the field of artificial intel-

979-8-3503-1597-4/23/$31.00 ©2023 IEEE

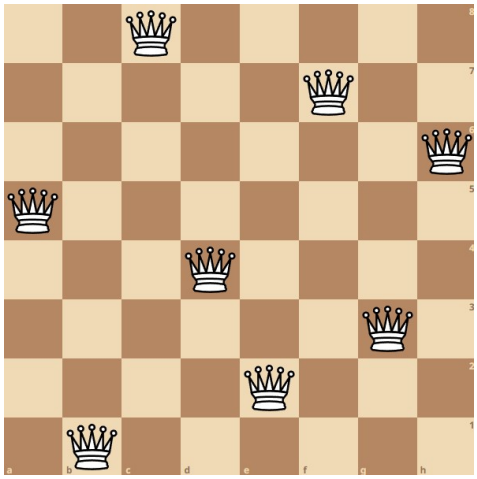


Fig. 2. Example of a valid board setting for 8-queen problem.

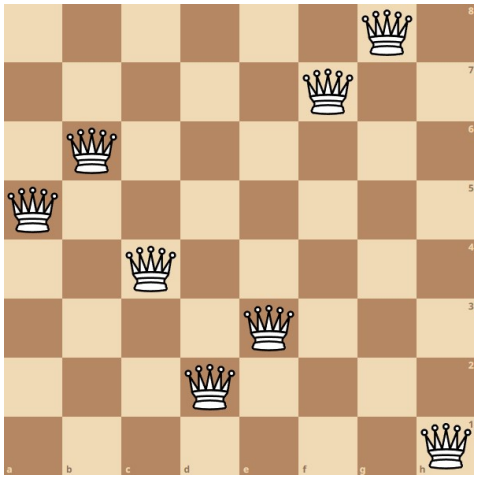


Fig. 3. The string '56423781' represents the setting of this board.

ligence. Due to its complexity, it works as an excellent test bench to evaluate the performance of newly discovered search algorithms and optimization techniques.

As mentioned earlier, there are 92 solutions for the 8-Queen problem. However, the number of possible solutions increase to 724 for 10 queens and to 14,200 for 12 queens. These figures indicate that the number of solutions grows exponentially as the puzzle size increases. This means that solving larger N-Queen problems requires an efficient algorithm that does not grow exponentially with the number of queens. It also shows that brute force methods like depth first and breadth first search will quickly be impractical to use once the number of queens exceeds beyond a certain number [2]–[4].

The complexity is the reason why the N-Queen problem remains popular and still attracts researchers' attention despite being around for so long [5]–[7]. It is also the reason why parallelization algorithms are usually applied to this problem as well [8]–[10]. Researchers use this problem to gain insights into the strengths and weaknesses of the latest search strategies (for example, of reinforcement learning), and compare those with existing algorithms [9], [11], [12]. The exponential growth in the search space with increasing puzzle size provides an ideal testbed for evaluating algorithm performance and scalability [13]–[15].

Researchers have employed various techniques [16] and modifications to existing algorithms to solve the N-Queen problem. For instance, one researcher proposed an algorithm that combined the minimal conflicts algorithm with a genetic algorithm to solve the puzzle [17], [18]. Another researcher utilized heuristic algorithms, such as simulated annealing and tabu search, to address the problem. A different study introduced a swapping algorithm that worked by exchanging the positions of the queens [19]–[21].

In yet another investigation, the speed of the genetic algorithm was enhanced by customizing the chromosome representation and evaluation function, resulting in a technique called Parallel Genetic Algorithm [8], [17], [22]. Another technology used for decentralized solutions to problems can be blockchain [23]. Other methods used to solve the N-Queen problem include Heuristic Genetic Algorithm [24], Differential Evolution, Covariance Matrix Adaptation Evolution Strategy [25], and Affine Boolean Function [26]. These diverse approaches demonstrate the ongoing interest in finding efficient solutions to this classic problem.

The objective of this paper is to solve the N-Queen problem using genetic algorithm and compare its performance with the two traditional search techniques - Breadth First Search (BFS) and Depth First Search (DFS) techniques. The study will then analyze which algorithm is suitable for different sizes of problems. Finally, the paper will examine the effect of different parameter values of the genetic algorithm to determine which values are most suitable for obtaining a quick and high-quality solution. While the work has been done in various studies at various levels, we intend to look at the recent advances in computational power and see upto which complexity can it be solved on the modern M1 microprocessors.

II. Methodology

In this work, a genetic algorithm is used to solve the N-Queen problem. Individuals, also known as chromosomes, are represented by a string of length N. Each character of the string represents the row-wise position of a queen, while the position of the character determines its column-wise position. For example, in the string '56423781', the character '6' is the second character of the string. This means that the queen is placed in the second column of the board, and the character '6' itself indicates that the queen is present in the sixth row of the chessboard. The length of the string informs us about the size of the board. The string '56423781' has a length of 8, which means it represents an 8 x 8 board. Figure 3 provides a graphical representation of the string '56423781'.

The fitness of a chromosome is calculated by a fitness function that returns the number of non-attacking pairs of queens on the board. The number of non-attacking pairs is calculated by subtracting the number of attacking pairs from

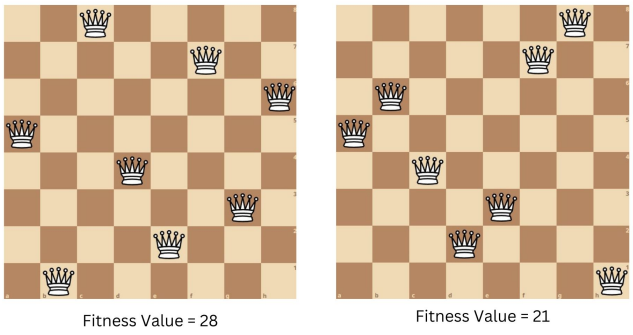


Fig. 4. Fitness values of two boards with 8 queens

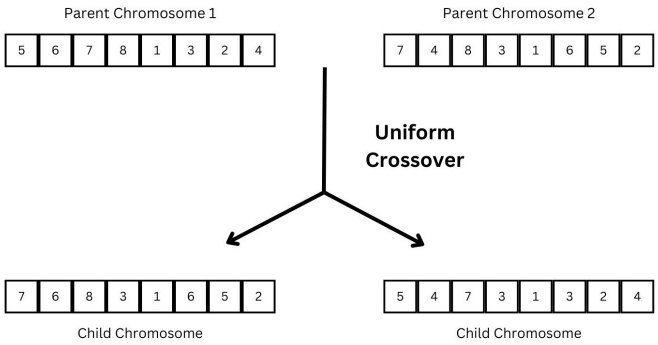


Fig. 5. Two possible offsprings resulting from uniform crossover.

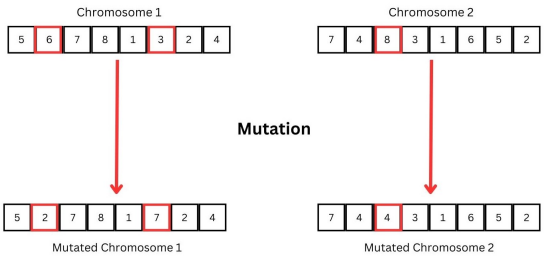


Fig. 6. Example of random resetting mutation.

the total number of pairs. An attacking pair is known as the conflict and conflicts are determined by a nested for-loop. If two queens are in the same row, column, or diagonal this is considered as a conflict. There are a total of NC2 pairs of queens. A string with a fitness value of NC2 is a perfect solution for the puzzle. For example for the 8-Queen problem, the maximum fitness value will be 8C2 which is equal to 28, this means that there is no queen on the boar is being attacked by any other queen. The fitness values of two chess boards with 8 queens is displayed for illustration in Figure 4.

The next step in the process of solving the N-Queen puzzle is the selection of parents for crossover. Tournament selection is used to select the fittest candidates from the current generation and these selected individuals are passed on to the next generation. A tournament size of 5 is kept, this means that 5 strings will be chosen from the generation and a tournament will be run amongst them. The winner of the tournament will be the string with the maximum fitness value. In the end, only the winner string will be selected as a parent for crossover. This process will continue until the desired amount of population is generated. At the end of the selection, we have a list of selected parent strings and now we are ready to perform crossover.

Now we have all the parents ready so we perform a uniform crossover. Uniform crossover means that each character of the child string is chosen from either of the parents with equal probability. Unlike point crossovers, parent chromosomes are not divided into segments in the uniform crossover. The main advantage of uniform crossover is that the newly generated will be completely different from the parent strings this will bring more diversity to our population and will increase our chances to get to the solution faster. Figure 5 shows two possible crossovers between two parents. Notice how the newly generated strings are completely different from the parent chromosomes and are also very divergent from each other.

After the crossover, there is the last component of the genetic algorithm which is mutation. This step is also inspired by the process of mutation in living organisms. The main reason behind this step is to bring diversity to the chromosomes. The mutation technique used in this research is known as random resetting mutation. We simply select a character we want to mutate and simply change the value to a random number from the range of available values. It is decided from the mutation rate whether to mutate a character or not. For this research mutation rate was set to 0.1 which means that 10% of the genes from a given chromosome will be mutated. Figure 6 shows how two different chromosomes are mutated. Notice that in chromosome 1 two genes are mutated and in chromosome 2 there is only one gene that was mutated. Also notice that mutated genes in both the chromosomes were on different positions.

In the experimental setup to evaluate the performance of the genetic algorithm on the N-Queen problem, we started by initializing a population. We generated 100 random chromosomes using a randomChromosome(N) function. This function took the size of chromosome, N, as input and returned a string of size N. We choose N = 8 to begin with. Each character in the randomly generated string was between 1 and 8. Once the strings were generated we started to iterate through the generations. In each generation, first, the parents were selected from the selection method. Parents were the fittest chromosomes from the existing population. Then crossover was performed between two adjacent parents in the parents' list. The chromosome generated by crossover was added to the offspring list. Then mutation method was applied to all the chromosomes in the offspring list. After the mutation step, a new population was created by merging the parents list with the offspring list. Then this newly generated population was

checked if it contained a perfect solution i.e. it contained a chromosome with a fitness value of 28. If the population contained the solution then iteration will stop otherwise we would iterate till the maximum number of generations is reached. These are known as the termination conditions. Figure 7 summarizes the whole setup for genetic algorithm.

The performance of the genetic algorithm will be compared to classical search strategies, BFS and DFS. The n-Queen problem was also solved using these algorithms as well. To implement the puzzle in BFS, a queen is placed on the board then all the possible children of that board are made and put into a queue as BFS explores the search space breadth-wise which means it will explore each level completely before moving on to the next level. This is an exhaustive search method so it continues to search until a result is found or all the possible states are explored.

The N-Queen problem is solved using DFS by exploring the search space depth-wise. Its implementation is the same as BFS except that it uses a stack instead of a queue. DFS also has the same termination condition as the BFS i.e., a solution is found or all the possibilities are explored. Experiments are carried out for different sizes of N-Queen problems and then the results of the three algorithms are compared.

III. Results

The N-Queen problem is solved using the genetic algorithm, breadth-first search, and depth-first search for 9 different sizes of problems, and the time taken by each algorithm to solve each problem is recorded in seconds. Experiments are done for the problems of size 1, 2, 4, 6, 8, 9, 10, 11, and 12 queens. It can be clearly seen in Table 1 that as the board size increases, the time taken by all three algorithms increases.

Board size	BFS (s)	DFS (s)	Genetic algorithm (s)
1	0.041	0.012	0.055
2	0.001	0.001	0.075
4	0.002	0.002	1.41
6	0.003	0.002	2.43
8	0.125	0.062	3.93
9	0.666	0.401	4.8
10	3.842	2.31	5.73
11	22.7	14.55	6.94
12	14.3	97.48	8.15

TABLE I
EXECUTION TIMES, IN SECONDS, FOR DIFFERENT ALGORITHMS AND BOARD SIZES

If we closely examine the table, it becomes apparent that for small-sized problems, BFS and DFS outperform the genetic algorithm. Problems with board sizes less than ten are considered small-sized problem instances. For instance, for the 8-Queen problem, BFS takes 0.125 seconds, DFS takes 0.0625 seconds, and the genetic algorithm takes 3.93 seconds. It can be concluded that the genetic algorithm solves the 8-Queen problem almost 60 times slower than the depth-first search method.

On the other hand, for medium and large sized-problems, where the board size is greater than 10, the genetic algorithm is way faster than the classical search approaches. For example for the board size of 12, BFS takes 143.3 seconds, DFS takes 97.5 seconds, and the genetic algorithm takes 8.2 seconds. It can be seen that the genetic algorithm is more than 10 times faster than the BFS and DFS.

From Figure 8, it can be observed that the execution time to solve the N-Queen problem for the genetic algorithm increases linearly as the board size increases. Meanwhile, time taken by BFS and DFS increases exponentially as the number of queens, on the board, are increased. It is also important to notice that DFS is faster than the BFS approach for all board sizes. The performance gap between the genetic algorithm and exhaustive search strategies widens significantly as the board size increases. These results suggest that the genetic algorithm is more scalable and performs far better than BFS and DFS for larger problems, while the depth-first search is best for small-sized problems.

IV. Discussion

The results of the experiment show that for small-sized puzzle searching strategies, BFS and DFS, perform better than the genetic algorithm. This is because there are few state spaces in smaller problems, so these search algorithms can explore all the states in less time to find the solutions. As the complexity of the problem increases and as the size of the board increases the state space increases also the number of possible solutions increases exponentially as a result execution time for searching algorithms also increases exponentially. On the other hand, the genetic algorithm is an optimization technique, and it doesn't explore all the states to reach the goal state, so its execution time increases linearly when the board's size increases.

Another important point to consider is that the genetic algorithm is an optimization technique, so it involves randomness that affects its performance. This is the reason why sometimes it is fast and sometimes it is slow for the same problem. The initial population generated is completely random, and if there is a solution already in the existing population or if there are parts of the solution available in the population, then the genetic algorithm will converge toward the goal state faster.

The setting of hyperparameters also affects the performance of the genetic algorithm. As we increase the mutation rate, the genetic algorithm converges to the solution faster. This is because a higher mutation rate means that more genes in a chromosome will be mutated which will diversify the population. This diversity will increase the chance for the solution to exist in the population thus resulting in a faster convergence rate towards the goal state.

As the genetic algorithm is an optimization technique so it can stuck in local minima or maxima, but this can be avoided with the aid of mutation. The genetic algorithm also guarantees to converge to a solution by the infinite monkey theorem given that the maximum number of generations is kept high enough.

It is also noticeable that DFS is faster than BFS for all sizes of problems. This is because it searches depth-wise which can be fast most of the time, but the BFS has its own advantages like it can never stuck into a cycle but DFS can.

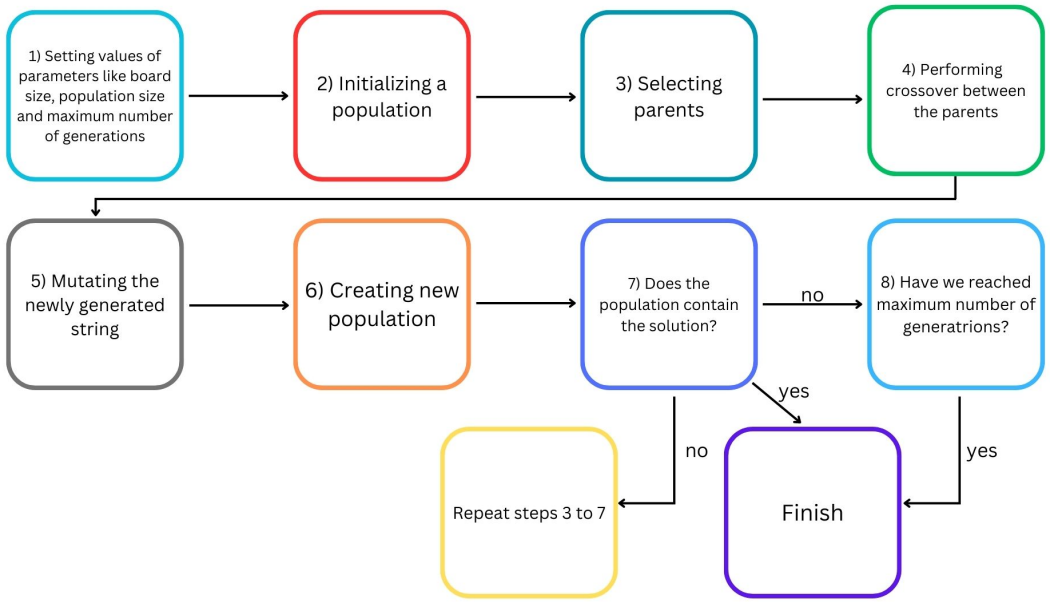


Fig. 7. The flowchart of complete methodology used to solve the N-Queen problem using genetic algorithm.

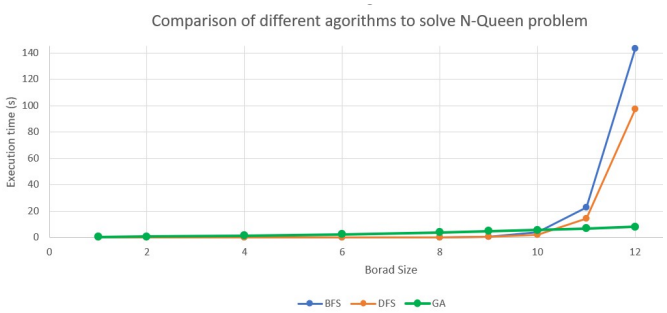


Fig. 8. Graph showing the execution times of different search algorithms for solving the N-Queen puzzle.

In this study, all the possible solutions were considered when experimenting. By "all possible solutions", we mean both isomorphic and non-isomorphic solutions were considered. Researchers have already worked with genetic algorithms and non-isomorphic solutions to the N-Queen problem [27]. In further research, comparisons between genetic algorithms and other search techniques can be made for non-isomorphic solutions only.

V. Conclusion

This study demonstrates that the genetic algorithm is best suited to solve N-Queens problems when the number of queens is greater than 10. For smaller problems with lesser number of queens, it is better to use the depth-first search algorithm, since the number of solutions increases exponentially, and the execution time for exhaustive search methods, such as BFS and DFS, will also increase exponentially, while the time taken for the genetic algorithm to converge to a solution increases linearly. The study also implies that the genetic algorithm will outperform other search techniques as the complexity of the problem increases. The genetic algorithm can be optimized by tuning the hyperparameters. Increasing the mutation rate will result in faster convergence towards a solution, as higher mutation rates will introduce more diversity into the population. It is also important to note that due to the randomness involved in the genetic algorithm, its performance may fluctuate.

References

[1] O. Aichholzer, D. Eppstein, and E.-M. Hainzl, "Geometric dominating sets-a minimum version of the no-three-in-line problem," *Computational Geometry*, vol. 108, p. 101913, 2023.

[2] A. Dehghani, R. Namvar, and A. Khalili, "Henry v: A linear time algorithm for solving the n-queens problem using only 5 patterns," in *2023 28th International Computer Conference, Computer Society of Iran (CSICC)*. IEEE, 2023, pp. 1–6.

[3] B. Z. El Abidine, "An incremental approach to the n-queen problem with polynomial time," *Journal of King Saud University-Computer and Information Sciences*, vol. 35, no. 3, pp. 1–7, 2023.

[4] A. Z. Ijaz, R. H. Ali, A. Sarwar, T. A. Khan, and M. M. Baig, "Importance of synteny in homology inference," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 234–239.

[5] P. Garg, S. S. Chauhan Gonder, and D. Singh, "Hybrid crossover operator in genetic algorithm for solving n-queens problem," in *Soft Computing: Theories and Applications: Proceedings of SoCTA 2021*. Springer, 2022, pp. 91–99.

[6] A. Z. Ijaz, R. H. Ali, N. Ali, T. Laique, and T. A. Khan, "Solving graph coloring problem via graph neural network (gnn)," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 178–183.

[7] M. H. Aslam, S. F. Hussain, and R. H. Ali, "Predictive analysis on severity of non-alcoholic fatty liver disease (nafld) using machine learning algorithms," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 95–100.

[8] C. Jianli, C. Zhikui, W. Yuxin, and G. He, "Parallel genetic algorithm for n-queens problem based on message passing interface-compute unified device architecture," *Computational Intelligence*, vol. 36, no. 4, pp. 1621–1637, 2020.

[9] P. Prudhvi Raj, P. Shah, and P. Suresh, "Faster convergence to n-queens problem using reinforcement learning," in *Modeling, Machine Learning and Astronomy: First International Conference, MMLA 2019, Bangalore, India, November 22–23, 2019, Revised Selected Papers 1*. Springer, 2020, pp. 66–77.

[10] A. A. Khan, R. H. Ali, and B. Mirza, "Evolutionary history of alzheimer disease-causing protein family presenilins with pathological implications," *Journal of Molecular Evolution*, vol. 88, no. 8-9, pp. 674–688, 2020.

[11] D. K. Mohammad, R. H. Ali, J. J. Turunen, B. F. Nore, and C. E. Smith, "B cell receptor activation predominantly regulates akt-mtorc1/2 substrates functionally related to rna processing," *PloS one*, vol. 11, no. 8, p. e0160255, 2016.

[12] Y. Yang, S. Tu, R. H. Ali, H. Alasmary, M. Waqas, and M. N. Amjad, "Intrusion detection based on bidirectional long short-term memory with attention mechanism," *Computers, Materials & Continua*, vol. 74, no. 1, pp. 801–815, 2023.

[13] M. Simkin, "The number of n-queens configurations," *Advances in Mathematics*, vol. 427, p. 109127, 2023.

[14] R. H. Ali, S. A. Muhammad, M. A. Khan, and L. Arvestad, "Quantitative synteny scoring improves homology inference and partitioning of gene families," in *BMC bioinformatics*, vol. 14, no. 15. BioMed Central, 2013, pp. 1–9.

[15] R. H. Ali and A. A. Khan, "Tracing the evolution of ferm domain of kindlins," *Molecular phylogenetics and evolution*, vol. 80, pp. 193–204, 2014.

[16] U. Arshad, M. Ali Shah, and N. Javaid, "Futuristic blockchain based scalable and cost-effective 5g vehicular network architecture," *Vehicular Communications*, vol. 31, p. 100386, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214209621000553

[17] J. E. A. Heris and M. A. Oskoei, "Modified genetic algorithm for solving n-queens problem," in *2014 Iranian Conference on Intelligent Systems (ICIS)*. IEEE, 2014, pp. 1–5.

[18] R. H. Ali, M. Bogusz, and S. Whelan, "Identifying clusters of high confidence homologies in multiple sequence alignments," *Molecular biology and evolution*, vol. 36, no. 10, pp. 2340–2351, 2019.

[19] N. Vaughan, "Swapping algorithm and meta-heuristic solutions for combinatorial optimization n-queens problem," in *2015 Science and Information Conference (SAI)*. IEEE, 2015, pp. 102–104.

[20] Z. ul Abideen, T. A. Khan, R. H. Ali, N. Ali, M. M. Baig, and M. S. Ali, "Docontap: Ai-based disease diagnostic system and recommendation system," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 148–153.

[21] N. Ali, S. Ansari, Z. Halim, R. H. Ali, M. F. Khan, and M. Khan, "Breast cancer classification and proof of key artificial neural network terminologies," in *2019 13th International Conference on Mathematics, Actuarial Science, Computer Science and Statistics (MACS)*. IEEE, 2019, pp. 1–6.

[22] N. Ali, M. R. Bonyadi, M. A. Raja, J. Akhtar, N. Javed, and C. Ryan, "Fuscd-future smart car driver," in *2019 22nd International Multitopic Conference (INMIC)*. IEEE, 2019, pp. 1–7.

[23] U. Arshad, S. Javaid, S. Ahmed, B. Seemab, and N. Javaid, "A futuristic blockchain based vehicular network architecture and trust management system," in *2019 International Conference on Advances in the Emerging Computing Technologies (AECT)*, 2020, pp. 1–6.

[24] K. Yang, J. Zheng, J. Zou, F. Yu, and S. Yang, "A dual-population evolutionary algorithm based on adaptive constraint strength for constrained multi-objective optimization," *Swarm and Evolutionary Computation*, p. 101247, 2023.

[25] J. Al-Afandi and A. Horváth, "Adaptive gene level mutation," *Algorithms*, vol. 14, no. 1, p. 16, 2021.

[26] R. K. Rout, J. K. Das, P. P. Choudhury *et al.*, "Solution of 16-queens problem using 3-variable affine boolean functions (2^{n+ 1})," in *2018 8th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 2018, pp. 853–855.

[27] S. T. Hedetniemi, "My top 10 graph theory conjectures and open problems," *Graph Theory: Favorite Conjectures and Open Problems-1*, pp. 109–134, 2016.

View publication stats